

Ejercicio Demostrativo

Luis Antonio Castillo Javier

Tiempo estimado: 45 minutos

Universidad San Carlos de Guatemala
Facultad de Ingeniería
Escuela de Ingeniería en Ciencias y Sistemas.
Segundo Semestre 2026
[Laboratorio - Introducción a la Programación y Computación 2 - Sección D]

1. Marco Formativo

1.1 Valores

Nombre del valor	¿Cómo se aplica en tu laboratorio?
Perseverancia	Durante el ejercicio demostrativo de ASP.NET Core Web API, la perseverancia se evidencia cuando el estudiante enfrenta errores de compilación o configuración del entorno .NET. Se espera que el estudiante documente los errores encontrados, investigue su causa y persista hasta obtener una API funcional, reflejando la actitud profesional esperada en el desarrollo de software.

1.2 Competencias

Tipo de Competencia	Redacción
Competencia General	Diseñar e implementar servicios backend mediante APIs REST utilizando frameworks modernos de desarrollo web, aplicando buenas prácticas de arquitectura de software.
Competencia Específica	Crear una Web API REST funcional usando ASP.NET Core, definiendo correctamente Modelos, Servicios y Controladores, y exponiéndola mediante métodos HTTP estándar (GET, POST, PUT, DELETE).

2. Palabras Clave

Backend: Parte del sistema que procesa la lógica, los datos y responde a las solicitudes del cliente.

API REST: Interfaz de programación que sigue los principios REST para comunicar sistemas a través del protocolo HTTP.

ASP.NET Core: Framework de Microsoft para crear aplicaciones y servicios web de alto rendimiento, multiplataforma.

Controller: Clase encargada de recibir solicitudes HTTP y devolver respuestas en una arquitectura Web API.

Model: Clase C# que define la estructura de datos que se maneja en la aplicación.

Service: Capa que contiene la lógica de negocio de la aplicación, separada del controlador.

JSON: Formato estándar para el intercambio de datos entre cliente y servidor.

Endpoint: URL específica dentro de una API que recibe solicitudes HTTP.

3. Resumen

Este ejercicio demostrativo presenta de forma guiada el desarrollo de una Web API REST básica utilizando ASP.NET Core Web API, framework de Microsoft. El propósito es mostrar al estudiante cómo aplicar los conceptos teóricos de backend, API REST y métodos HTTP en un caso concreto y funcional.

La clase se orienta a que el estudiante comprenda la estructura en capas de una Web API (Model, Service, Controller), identifique la secuencia de pasos para crear y exponer endpoints funcionales, relacione la práctica con los contenidos de la Semana 10 del curso, y reconozca la forma correcta de verificar que la API responde adecuadamente mediante herramientas de prueba como un navegador o cliente HTTP.

4. Contenido

Descripción del ejercicio

El ejercicio consiste en crear una Web API REST sencilla en ASP.NET Core que gestione una lista de estudiantes. Se parte desde la creación del proyecto hasta la exposición de endpoints para consultar, agregar, actualizar y eliminar estudiantes. El resultado esperado al finalizar es una API funcional que responda correctamente a solicitudes HTTP con datos en formato JSON.

Objetivo de la demostración

Demostrar el procedimiento para crear una Web API REST funcional, utilizando ASP.NET Core Web API y la arquitectura Model-Service-Controller, con el fin de que el estudiante comprenda cómo estructurar y exponer servicios backend accesibles mediante métodos HTTP estándar.

Recursos necesarios

- Software: .NET SDK 8.0 o superior, Visual Studio 2022 o Visual Studio Code.
- Herramientas técnicas: Postman o Thunder Client para probar los endpoints; terminal/consola del sistema.
- Archivos o materiales de apoyo: Presentación de Semana 10, documentación oficial de ASP.NET Core (docs.microsoft.com).
- Equipo necesario: Computadora con al menos 8 GB de RAM y conexión a internet.
- Configuración mínima: .NET SDK instalado y verificado con el comando `dotnet --version`.

Desarrollo paso a paso

Nota: Describe el procedimiento en orden lógico y operativo. Esta guía está pensada para ser utilizada por otro tutor en semestres posteriores, por lo que cada paso debe ser claro, reproducible y sin ambigüedades.

Paso 1 — Crear el proyecto ASP.NET Core Web API

Abrir la terminal y ejecutar el siguiente comando para crear el proyecto:

```
dotnet new webapi -n EstudiantesAPI
```

Ingresar a la carpeta del proyecto con `cd EstudiantesAPI`. El objetivo de este paso es generar la estructura base del proyecto, que incluye un controlador de ejemplo (WeatherForecastController) que el estudiante debe observar para entender la anatomía de un controlador en ASP.NET Core.

Paso 2 — Crear el Modelo

Dentro del proyecto, crear una carpeta llamada Models y dentro de ella un archivo Estudiante.cs con la siguiente clase:

```
public class Estudiante { public int Id { get; set; } public string Nombre { get; set; } public string Carnet { get; set; } }
```

Este modelo define la estructura de datos que se manejará en la API. El estudiante debe observar que en ASP.NET Core un modelo es simplemente una clase C# con propiedades. Antes de continuar verificar que el archivo compila sin errores con dotnet build.

Paso 3 — Crear el Servicio

Crear la carpeta Services y dentro el archivo EstudianteService.cs. Este servicio usará una lista en memoria para simular la base de datos. Implementar métodos para: ObtenerTodos(), ObtenerPorId(int id), Agregar(Estudiante e), Actualizar(Estudiante e) y Eliminar(int id). Registrar el servicio en Program.cs con: builder.Services.AddSingleton<EstudianteService>(). El estudiante debe verificar que el servicio queda registrado correctamente antes de pasar al controlador.

Paso 4 — Crear el Controlador y probar los endpoints

Dentro de la carpeta Controllers crear EstudiantesController.cs. Decorar la clase con [ApiController] y [Route("api/[controller]")]. Inyectar el servicio mediante el constructor. Implementar los métodos GET, POST, PUT y DELETE usando los atributos [HttpGet], [HttpPost], [HttpPut], [HttpDelete]. Ejecutar la API con dotnet run y abrir la URL indicada en la consola. Usar Postman para hacer una solicitud GET a /api/estudiantes y verificar que retorna un arreglo JSON. Si la respuesta es 200 OK con datos, el ejercicio fue resuelto correctamente.

Observaciones importantes

- Asegurarse de que el SDK de .NET esté instalado antes de comenzar. Ejecutar dotnet --version para verificar.
- El orden de creación es importante: primero el Modelo, luego el Servicio, finalmente el Controlador.
- Error común: olvidar registrar el servicio en Program.cs, lo cual genera una excepción de inyección de dependencias al iniciar la API.
- En ASP.NET Core 8, el archivo de configuración principal es Program.cs (no Startup.cs como en versiones anteriores).
- Recordar que cada endpoint debe ser probado de forma independiente para confirmar su correcto funcionamiento.

Tabla 1 — Métodos HTTP y su función en la API de Estudiantes

Método HTTP	Endpoint	Acción	Respuesta esperada
GET	/api/estudiantes	Obtener todos los estudiantes	200 OK + JSON array
POST	/api/estudiantes	Agregar un estudiante nuevo	201 Created + JSON objeto

PUT	/api/estudiantes/{id}	Actualizar datos de un estudiante	200 OK o 204 No Content
DELETE	/api/estudiantes/{id}	Eliminar un estudiante por ID	200 OK o 404 Not Found

Resultado esperado

Al finalizar el ejercicio, el estudiante debe tener en ejecución una Web API REST en ASP.NET Core que exponga al menos 4 endpoints funcionales (GET, POST, PUT, DELETE) para la entidad Estudiante. La API debe responder correctamente con datos en formato JSON y con códigos HTTP apropiados (200, 201, 404) al ser consultada desde Postman o cualquier cliente HTTP. El proyecto debe compilar sin errores y la arquitectura debe respetar la separación Model-Service-Controller.

Aplicación práctica

Este ejercicio constituye la base para cualquier proyecto de software moderno que requiera un backend. La arquitectura Model-Service-Controller aprendida en ASP.NET Core es directamente aplicable en entornos profesionales de desarrollo .NET. Además, los conceptos de API REST y métodos HTTP son transferibles a otros frameworks como Flask (Python), Spring Boot (Java) o Express (Node.js), lo que hace de este conocimiento una habilidad fundamental para el desarrollo full-stack.

5. Aconsejando al siguiente Tutor para la realización de ejemplos

- Dificultades encontradas: El principal obstáculo fue la configuración inicial del entorno de desarrollo. Varios estudiantes no tenían el .NET SDK instalado correctamente. Se recomienda verificar la instalación al inicio de la clase con `dotnet --version` antes de comenzar cualquier demo.
- Recomendaciones para futuras sesiones: Tener el proyecto base ya creado como respaldo en caso de problemas de conectividad o tiempo. Separar la demo en dos partes si el tiempo es ajustado: Parte 1 (Model y Service) y Parte 2 (Controller y pruebas).
- Puntos de mayor dificultad para el estudiante: La inyección de dependencias fue el concepto más difícil de asimilar. Se recomienda dedicar 5 minutos adicionales a explicar por qué se registra el servicio en `Program.cs` y cómo ASP.NET Core lo provee automáticamente al controlador mediante el constructor.

6. Referencias

1. Microsoft. (2024). Tutorial: Create a web API with ASP.NET Core. Microsoft Learn. <https://learn.microsoft.com/en-us/aspnet/core/tutorials/first-web-api>
2. Platzi. (2018, febrero 21). Qué es Frontend y Backend: principal diferencia y ejemplos. <https://platzi.com/blog/que-es-frontend-y-backend/>
3. Lucena, P. (2023, mayo 6). ¿Qué es el framework? Maestrías y MBA. <https://www.cesuma.mx/blog/que-es-el-framework.html>
4. MDN Web Docs. (2024). HTTP request methods. Mozilla Developer Network. <https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods>